

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Sun Oct 5 08:07:40 2025

@author: sebastienanard
"""

import numpy as np
import random
import hashlib
import secrets
from sympy import isprime

def generate_prime(bits):
    while True:
        num = random.getrandbits(bits)
        if isprime(num):
            return num

def generate_keys(bits):

    # Génération de p et q (grands nombres premiers)
    p = generate_prime(bits//2)
    q = generate_prime(bits//2)
    while p == q:
        q = generate_prime(bits)

    # Calcul de n et  $\phi(n)$ 
    n = p * q
    phi_n = (p - 1) * (q - 1)

    # Choix aléatoire de e
    e = generate_prime(bits)
    # e = generate_prime(bits*2) Version avec génération aléatoire
    # Calcul de d comme inverse modulaire de e modulo  $\phi(n)$ 
    d = pow(e, -1, phi_n)

    # Clé publique (n, e) et clé privée (n, d)
    return (n, e), (n, d)

def bytes_to_int(byte_list):
    return int.from_bytes(bytes(byte_list), byteorder='big', signed=False)

def int_to_bytes(entier, length):
    return list(int.to_bytes(entier, length=length, byteorder='big', signed=False))

def encrypt_rsa(message, public_key):
    """Chiffre un message avec RSA et la clé publique (n, e)."""
    n, e = public_key
    message_int = bytes_to_int(message)
    cipher_int = pow(message_int, e, n)
    return cipher_int

def decrypt_rsa(cipher_int, private_key):
    n, d = private_key
    mod_bytes = (n.bit_length() + 7) // 8
    message_int = pow(cipher_int, d, n)
    return int_to_bytes(message_int, mod_bytes)

def pkcs7_pad(data, block_size=32):
    """
    Pads the given plaintext with PKCS#7 padding to a multiple of 16 bytes.
    Note that if the plaintext size is a multiple of 16,
    a whole block will be added.
    """
    padding_len = block_size - (len(data) % block_size)
    padding = [padding_len] * padding_len
    return data + padding

```

```

def pkcs7_unpad(data, block_size=32):
    """
    Removes a PKCS#7 padding, returning the unpadded text and ensuring the
    padding was correct.
    """
    padding_len = data[-1]
    return data[:-padding_len]

def hash_data(data, hash_algo=hashlib.sha256):
    return hash_algo(data).digest()

def get_hash_length(hash_algo=hashlib.sha256):
    return hash_algo().digest_size

def oaep_encode(message, hash_algo=hashlib.sha256):
    h_len = get_hash_length(hash_algo)

    message = pkcs7_pad(message)

    seed = secrets.token_bytes(h_len)

    # Assurez-vous que s_mask a la même longueur que message
    s_mask = hash_data(seed, hash_algo)
    s = bytes([message[i] ^ s_mask[i % len(s_mask)] for i in range(h_len)])

    # Assurez-vous que t_mask a la même longueur que seed
    t_mask = hash_data(s, hash_algo)
    t = bytes([seed[i] ^ t_mask[i % len(t_mask)] for i in range(h_len)])

    encoded = t + s

    return encoded

def oaep_decode(encoded, hash_algo=hashlib.sha256):
    h_len = get_hash_length(hash_algo)

    size_encoded = len(encoded)

    t = encoded[size_encoded-2*h_len: size_encoded-h_len]
    s = encoded[size_encoded-h_len:size_encoded]

    # Assurez-vous que seed_mask a la même longueur que t
    seed_mask = hash_data(bytes(s), hash_algo)
    seed = bytes([t[i] ^ seed_mask[i % len(seed_mask)] for i in range(h_len)])

    # Assurez-vous que m_mask a la même longueur que s
    m_mask = hash_data(seed, hash_algo)
    message = bytes([s[i] ^ m_mask[i % len(m_mask)] for i in range(len(s))])

    return pkcs7_unpad(message)

def encrypt_rsa_oaep(message, public_key):
    padded_message = oaep_encode(message)
    cipher = encrypt_rsa(padded_message, public_key)
    return cipher

def decrypt_rsa_oaep(cipher, private_key):
    padded_message = decrypt_rsa(cipher, private_key)
    message = oaep_decode(padded_message)
    return list(message)

# La matrice S-box utilisée dans SubBytes
S_BOX = np.array([
    [0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab,
    0x76],
    [0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72,

```

```

0xc0],
[0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31,
0x15],
[0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2,
0x75],
[0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f,
0x84],
[0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58,
0xcf],
[0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f,
0xa8],
[0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3,
0xd2],
[0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19,
0x73],
[0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b,
0xdb],
[0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4,
0x79],
[0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae,
0x08],
[0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b,
0x8a],
[0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d,
0x9e],
[0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28,
0xdf],
[0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb,
0x16]
], dtype=np.uint8)

```

La matrice inverse Inv S-Box utilisée en déchiffrement

```

INV_S_BOX = np.array([
[0x52, 0x09, 0x6A, 0xD5, 0x30, 0x36, 0xA5, 0x38, 0xBF, 0x40, 0xA3, 0x9E, 0x81, 0xF3, 0xD7,
0xFB],
[0x7C, 0xE3, 0x39, 0x82, 0x9B, 0x2F, 0xFF, 0x87, 0x34, 0x8E, 0x43, 0x44, 0xC4, 0xDE, 0xE9,
0xCB],
[0x54, 0x7B, 0x94, 0x32, 0xA6, 0xC2, 0x23, 0x3D, 0xEE, 0x4C, 0x95, 0x0B, 0x42, 0xFA, 0xC3,
0x4E],
[0x08, 0x2E, 0xA1, 0x66, 0x28, 0xD9, 0x24, 0xB2, 0x76, 0x5B, 0xA2, 0x49, 0x6D, 0x8B, 0xD1,
0x25],
[0x72, 0xF8, 0xF6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xD4, 0xA4, 0x5C, 0xCC, 0x5D, 0x65, 0xB6,
0x92],
[0x6C, 0x70, 0x48, 0x50, 0xFD, 0xED, 0xB9, 0xDA, 0x5E, 0x15, 0x46, 0x57, 0xA7, 0x8D, 0x9D,
0x84],
[0x90, 0xD8, 0xAB, 0x00, 0x8C, 0xBC, 0xD3, 0x0A, 0xF7, 0xE4, 0x58, 0x05, 0xB8, 0xB3, 0x45,
0x06],
[0xD0, 0x2C, 0x1E, 0x8F, 0xCA, 0x3F, 0x0F, 0x02, 0xC1, 0xAF, 0xBD, 0x03, 0x01, 0x13, 0x8A,
0x6B],
[0x3A, 0x91, 0x11, 0x41, 0x4F, 0x67, 0xDC, 0xEA, 0x97, 0xF2, 0xCF, 0xCE, 0xF0, 0xB4, 0xE6,
0x73],
[0x96, 0xAC, 0x74, 0x22, 0xE7, 0xAD, 0x35, 0x85, 0xE2, 0xF9, 0x37, 0xE8, 0x1C, 0x75, 0xDF,
0x6E],
[0x47, 0xF1, 0x1A, 0x71, 0x1D, 0x29, 0xC5, 0x89, 0x6F, 0xB7, 0x62, 0x0E, 0xAA, 0x18, 0xBE,
0x1B],
[0xFC, 0x56, 0x3E, 0x4B, 0xC6, 0xD2, 0x79, 0x20, 0x9A, 0xDB, 0xC0, 0xFE, 0x78, 0xCD, 0x5A,
0xF4],
[0x1F, 0xDD, 0xA8, 0x33, 0x88, 0x07, 0xC7, 0x31, 0xB1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xEC,
0x5F],
[0x60, 0x51, 0x7F, 0xA9, 0x19, 0xB5, 0x4A, 0x0D, 0x2D, 0xE5, 0x7A, 0x9F, 0x93, 0xC9, 0x9C,
0xEF],
[0xA0, 0xE0, 0x3B, 0x4D, 0xAE, 0x2A, 0xF5, 0xB0, 0xC8, 0xEB, 0xBB, 0x3C, 0x83, 0x53, 0x99,
0x61],
[0x17, 0x2B, 0x04, 0x7E, 0xBA, 0x77, 0xD6, 0x26, 0xE1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0C,
0x7D]
], dtype=np.uint8)

```

Transformations SubBytes

```
def sub_bytes(state):
```

```

return np.vectorize(lambda x: S_BOX[x >> 4][x & 0x0F])(state)

def inv_sub_bytes(state):
    return np.vectorize(lambda x: INV_S_BOX[x >> 4][x & 0x0F])(state)

# Transformations ShiftRows
def shift_rows(state):
    new_state = np.copy(state)
    for i in range(4):
        new_state[i] = np.roll(new_state[i], -i)
    return new_state

def inv_shift_rows(state):
    new_state = np.copy(state)
    for i in range(4):
        new_state[i] = np.roll(new_state[i], i)
    return new_state

# Transformations MixColumns
def mix_columns(state):
    for i in range(4):
        col = state[:, i]
        state[:, i] = [
            gmul(col[0], 2) ^ gmul(col[1], 3) ^ col[2] ^ col[3],
            col[0] ^ gmul(col[1], 2) ^ gmul(col[2], 3) ^ col[3],
            col[0] ^ col[1] ^ gmul(col[2], 2) ^ gmul(col[3], 3),
            gmul(col[0], 3) ^ col[1] ^ col[2] ^ gmul(col[3], 2)
        ]
    return state

def inv_mix_columns(state):
    for i in range(4):
        col = state[:, i]
        state[:, i] = [
            gmul(col[0], 0x0e) ^ gmul(col[1], 0x0b) ^ gmul(col[2], 0x0d) ^ gmul(col[3], 0x09),
            gmul(col[0], 0x09) ^ gmul(col[1], 0x0e) ^ gmul(col[2], 0x0b) ^ gmul(col[3], 0x0d),
            gmul(col[0], 0x0d) ^ gmul(col[1], 0x09) ^ gmul(col[2], 0x0e) ^ gmul(col[3], 0x0b),
            gmul(col[0], 0x0b) ^ gmul(col[1], 0x0d) ^ gmul(col[2], 0x09) ^ gmul(col[3], 0x0e)
        ]
    return state

# Fonction galois pour multiplication dans MixColumns
def gmul(a, b):
    p = 0
    for _ in range(8):
        if b & 1:
            p ^= a
        hi_bit_set = a & 0x80
        a = (a << 1) & 0xFF
        if hi_bit_set:
            a ^= 0x1B
        b >>= 1
    return p

# Transformation AddRoundKey
def add_round_key(state, round_key):
    return np.bitwise_xor(state, round_key)

# Calcul de la clé étendue
NB = 4 # Nombre de colonnes dans l'état (4 pour AES)
NK = 4 # Nombre de mots dans la clé d'entrée (4 pour AES-128)
NR = 10 # Nombre de rondes pour AES-128 (10)

# La table des constantes Rcon pour chaque tour
RCON = [
    0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36
]

```

```

# Quelques fonctions additionnelles
def sub_word(word):
    """Applique la substitution S-Box sur chaque octet du mot."""
    return [S_BOX[b >> 4][b & 0x0F] for b in word]

def rot_word(word):
    """Rotation circulaire d'un mot de 4 octets (utilisé dans la clé étendue)."""
    return word[1:] + word[:1]

# Calcul effectif de la clé étendue
def expand_key(key, rounds):
    """Génère la clé étendue pour AES."""
    key_symbols = list(key)
    key_schedule = [key_symbols[i * 4:(i + 1) * 4] for i in range(NK)]

    for i in range(NK, NB * (rounds + 1)):
        temp = key_schedule[i - 1]

        if i % NK == 0:
            temp = sub_word(rot_word(temp))
            temp[0] ^= RCON[(i // NK) - 1]

        # Ajout XOR avec le mot NK positions plus tôt
        key_schedule.append([
            key_schedule[i - NK][j] ^ temp[j] for j in range(4)
        ])

    # Transformation en une liste d'octets
    expanded_key = [item for sublist in key_schedule for item in sublist]
    return np.array(expanded_key).reshape(-1, 4, 4)

# Fonction de chiffrement
def encrypt(plaintext, key, rounds):
    # Initialisation de l'état
    state = np.array(plaintext, dtype=np.uint8).reshape(4, 4)
    round_keys = expand_key(key, rounds)

    state = add_round_key(state, round_keys[0])
    for round in range(1, rounds):
        state = sub_bytes(state)
        state = shift_rows(state)
        state = mix_columns(state)
        state = add_round_key(state, round_keys[round])

    # Dernière ronde sans MixColumns
    state = sub_bytes(state)
    state = shift_rows(state)
    state = add_round_key(state, round_keys[rounds])

    return state.flatten().tolist()

# Fonction de déchiffrement
def decrypt(ciphertext, key, rounds):
    """Déchiffrement du texte chiffré AES."""
    # Initialisation de l'état
    state = np.array(ciphertext, dtype=np.uint8).reshape(4, 4)
    round_keys = expand_key(key, rounds)

    # Étape initiale : AddRoundKey avec la dernière sous-clé
    state = add_round_key(state, round_keys[rounds])

    # Boucle de déchiffrement sur les tours
    for round in range(rounds - 1, 0, -1):
        state = inv_shift_rows(state)
        state = inv_sub_bytes(state)
        state = add_round_key(state, round_keys[round])
        state = inv_mix_columns(state)

    # Dernière étape : inv_shift_rows, inv_sub_bytes et AddRoundKey

```

```

state = inv_shift_rows(state)
state = inv_sub_bytes(state)
state = add_round_key(state, round_keys[0])

return state.flatten().tolist()

def xor_texts(text1, text2):
    """
    Retourne le XOR de deux textes en clair de même longueur.
    """
    return [b1 ^ b2 for b1, b2 in zip(text1, text2)]

def xor_blocks(block1, block2):
    """
    Applique l'opération XOR entre deux blocs de même longueur.
    """
    return [b1 ^ b2 for b1, b2 in zip(block1, block2)]

def cbc_mac(plaintext, key, iv, rounds=10):
    """
    Chiffre le plaintext en mode CBC avec padding PKCS#7.
    """
    # Ajoute du padding au plaintext
    padded_plaintext = pkcs7_pad(plaintext)
    #mac = []
    previous_block = iv

    # Découpe le plaintext en blocs de 16 octets
    for i in range(0, len(padded_plaintext), 16):
        block = padded_plaintext[i:i+16]

        # XOR avec le bloc précédent (ou IV pour le premier bloc)
        block = xor_blocks(block, previous_block)

        # Chiffrement du bloc
        encrypted_block = encrypt(block, key, rounds)

        # Ajoute le bloc chiffré au ciphertext
        #mac.extend(encrypted_block)

        # Met à jour le bloc précédent pour le prochain tour
        previous_block = encrypted_block

    return encrypted_block

def increment_counter(counter):
    """
    Incrmente un compteur de 16 octets (en supposant un compteur 128 bits).
    """
    counter_int = int.from_bytes(counter, byteorder="big") + 1
    return list(counter_int.to_bytes(16, byteorder="big"))

def ctr_crypt(data, key, nonce, rounds=10):
    """
    Opération CTR de base.
    """
    block_size = 16
    encrypted_data = []
    counter = nonce + [0] * 8 # Initialisation du compteur (nonce + 8 octets de compteur)

    # Traite les données bloc par bloc de 16 octets
    for i in range(0, len(data), block_size):
        # Génère le keystream en chiffrant le compteur actuel
        keystream_block = encrypt(counter, key, rounds)

        # XOR chaque octet du bloc de données avec le keystream

```

```

        block = data[i:i+block_size]
        encrypted_block = xor_blocks(block, keystream_block[:len(block)])
        encrypted_data.extend(encrypted_block)

        # Incrémente le compteur pour le prochain bloc
        counter = increment_counter(counter)

    return encrypted_data

def ctr_encrypt(data, key, nonce, rounds=10):
    """
    Chiffre les données en mode CTR (même opération pour les deux).
    """
    data = pkcs7_pad(data)
    return ctr_crypt(data, key, nonce)

def ctr_decrypt(data, key, nonce, rounds=10):
    """
    Chiffre les données en mode CTR (même opération pour les deux).
    """
    plaintext = ctr_crypt(data, key, nonce)
    return pkcs7_unpad(plaintext)

"""
Question 3 - Encapsulation
"""

print("-----")
print("Partie 3 - Encapsulation")
print("-----")

"""
Partie initiale de Bob
"""

""" Génération des clés RSA """
public_key_Bob, private_key_Bob = generate_keys(bits=2048)
n_Bob, e_Bob = public_key_Bob

"""
Partie d'Alice'
"""

""" Génération des clés AES """
key_mac_Alice = [0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6, 0xab, 0xf7, 0xcf, 0x80, 0x09,
0x89, 0x01, 0x0c]
key_enc_Alice = [0x64, 0xa7, 0x08, 0x23, 0xfe, 0xea, 0x86, 0xe1, 0xba, 0x7f, 0x09, 0x56, 0x1a,
0xbb, 0x00, 0xca]
iv = [0x04] * 16 # Exemple d'IV de 16 octets
nonce = [0x03] * 8 # Exemple de nonce de 8 octets

""" Choix d'un message initial """
plaintext_Alice = [0x32, 0x43, 0xf6, 0xa8, 0x88, 0x5a, 0x30, 0x8d, 0x31, 0x31, 0x98, 0xa2, 0xe0,
0x37, 0x07, 0x34, 0x21, 0x2a, 0x11, 0x9e, 0x1a, 0x00, 0x81, 0x50, 0xb1, 0xe5, 0xee, 0x77, 0x12,
0xe7, 0x70, 0x28]
print("Message initial d'Alice :", plaintext_Alice, "\n")

""" Chiffrement du message initial en AES-CTR """
ciphertext = ctr_encrypt(plaintext_Alice, key_enc_Alice, nonce, rounds=10)

""" Calcul du MAC sur le message initial avec AES-CBC """
mac_Alice = cbc_mac(ciphertext, key_mac_Alice, iv, rounds=10)
print("MAC calculé par Alice :", mac_Alice, "\n")

""" Chiffrement des clés MAC et ENC en RSA-OAEP par Alice """
cipher_mac = encrypt_rsa_oaep(key_mac_Alice, public_key_Bob)
cipher_enc = encrypt_rsa_oaep(key_enc_Alice, public_key_Bob)

```

```
"""
```

```
Partie de Bob
```

```
"""
```

```
""" Déchiffrement des clés MAC et ENC par Bob """
```

```
key_mac_Bob = decrypt_rsa_oaep(cipher_mac, private_key_Bob)
```

```
key_enc_Bob = decrypt_rsa_oaep(cipher_enc, private_key_Bob)
```

```
""" Vérification du MAC par Bob """
```

```
mac_Bob = cbc_mac(ciphertext, key_mac_Bob, iv, rounds=10)
```

```
print("MAC calculé par Bob :", mac_Bob, "\n")
```

```
""" Déchiffrement et affichage du message par Bob """
```

```
decrypted_text = ctr_decrypt(ciphertext, key_enc_Bob, nonce, rounds=10)
```

```
print("Texte déchiffré par Bob :", decrypted_text, "\n")
```